Database development with PL/SQL INSY 8311

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | eric.maniraguha@auca.ac.rw | LinkedIn Profile

6h00 pm – 9h50 pm

- Monday A -G207
- Tuesday B-G204
- Wednesday E-106
- Thursday F-G307

March 2025

Database development with PL/SQL

Reference reading

- 5 PL/SQL Collections and Records
- YouTube Tutorial: oracle plsql records and collections type

Lecture 07 – PLSQL Collections and Records

Objectives of PL/SQL Collections and Records

PL/SQL **Collections and Records** provide efficient ways to handle and manage structured data within PL/SQL programs. The key objectives of using them include:

Objectives of PL/SQL Collections and Records

1. Handling Multiple Values in a Single Variable

- Unlike scalar variables that hold only one value at a time, collections allow storing multiple values in a structured format.

2. Improved Performance

- Collections enable bulk processing, reducing the number of context switches between PL/SQL and SQL engines, which enhances performance.

3. Ease of Data Manipulation

- Collections support various methods to manipulate data efficiently, such as extending, trimming, and deleting elements dynamically.

4. Structured Data Storage

- Collections provide a way to store related data in a structured manner, similar to arrays or lists in other programming languages.

5. Efficient Query Handling

- Collections can be used to retrieve and manipulate sets of rows from queries without requiring multiple database round trips.

6. Flexibility in Size and Indexing

- Some collection types (like VARRAYs) have a predefined limit, while others (like Nested Tables and Associative Arrays) allow dynamic growth.

7. Passing Complex Data Between Procedures

- Collections allow passing multiple values between procedures and functions efficiently, reducing the need for multiple parameters.

8. Support for Bulk Operations

- With bulk processing (BULK COLLECT and FORALL), collections improve the efficiency of DML operations.

Types of Collections in PL/SQL

1. **Associative Arrays (Index-by Tables)** – Key-value pairs with flexible indexing.
2. **Nested Tables** – Unbounded, can store multiple rows, useful for bulk data processing.
3. **VARRAYs (Variable-Size Arrays)** – Fixed-size, ordered collections.

PL/SQL Collections and Records Overview

PL/SQL provides **composite data types**, which allow storing multiple values in a single variable. These types include:

1. **Collections**: A set of elements with the same data type.
2. **Records**: A set of fields that can have different data types.

Collections in PL/SQL

A **collection** is a data structure that holds multiple elements of the same type, and it allows access through indexing.

Types of Collections

PL/SQL supports **three** types of collections:

Collection Type	Number of Elements	Index Type	Dense or Sparse	Where Defined
Associative Array	Unspecified	String or PLS_INTEGER	Either	PL/SQL block/package
VARRAY (Variable-Size Array)	Specified	Integer	Always dense	PL/SQL block/package/schema
Nested Table	Unspecified	Integer	Starts dense, can become sparse	PL/SQL block/package/schema

Collections: Associative Arrays & Varray

Associative Arrays

- Also called **index-by tables**.
- Indexed using **PLS_INTEGER** or **VARCHAR2**.
- Stores **key-value pairs**.
- **Unbounded** (no fixed size).
- Does **not require disk space**.

Associative Arrays

```
DECLARE
  -- Declare an associative array (index-by table) type called 'population'
  -- The array stores numbers (population values) indexed by VARCHAR2 (city names)
  TYPE population IS TABLE OF NUMBER INDEX BY VARCHAR2(64);

  -- Declare a variable 'city_population' of type 'population'
  city_population population;
BEGIN
  -- Assign population values to specific cities
  city_population('Kigali') := 1200000;
  city_population('Musanze') := 450000;

  -- Output the population of Kigali
  DBMS_OUTPUT.PUT_LINE('Population of Kigali: ' || city_population('Kigali'));
END;
/
```

Output

```
Population of Kigali: 1200000
```

VARRAY (Variable-Size Array)

- **Fixed upper bound**.
- Always **dense** (no gaps).
- Stored **in-line** with the PL/SQL block.

Varray

```
DECLARE
  -- Declare a VARRAY type named NumArray that can hold up to 5 NUMBER values
  TYPE NumArray IS VARRAY(5) OF NUMBER;

  -- Declare a variable 'nums' of type NumArray and initialize it with three values
  nums NumArray := NumArray(10, 20, 30);

  -- Variable to store the sum of elements
  total NUMBER := 0;
BEGIN
  -- Display all elements in the collection
  DBMS_OUTPUT.PUT_LINE('Elements in the collection:');

  -- Loop through the elements of the VARRAY
  FOR i IN 1 .. nums.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE('Element ' || i || ': ' || nums(i));
    -- Add each element to the total sum
    total := total + nums(i);
  END LOOP;

  -- Display the sum of all elements
  DBMS_OUTPUT.PUT_LINE('Sum of elements: ' || total);
END;
/
```

Output

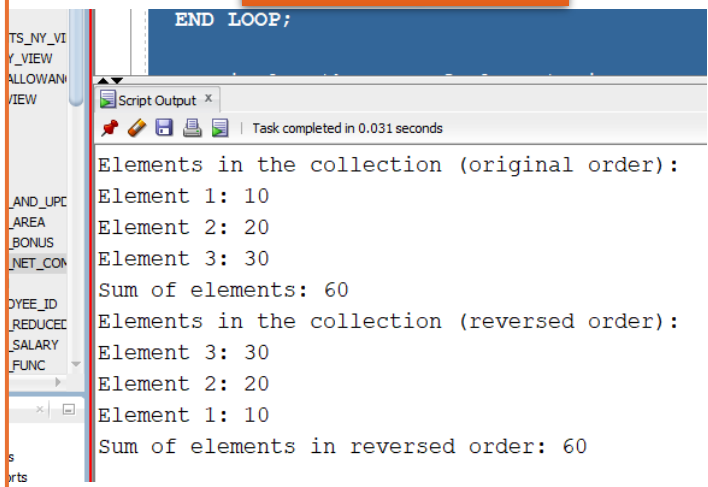
```
Elements in the collection:
Element 1: 10
Element 2: 20
Element 3: 30
Sum of elements: 60
```

VARRAY (Variable-Size Array)

To reverse the elements in the VARRAY

```
DECLARE
  -- Declare a VARRAY type named NumArray that can hold up to 5 NUMBER values
  TYPE NumArray IS VARRAY(5) OF NUMBER;

  -- Declare a variable 'nums' of type NumArray and initialize it with three values
  nums NumArray := NumArray(10, 20, 30);

  -- Variable to store the sum of elements
  total NUMBER := 0;
BEGIN
  -- Display all elements in the collection
  DBMS_OUTPUT.PUT_LINE('Elements in the collection (original order):');

  -- Loop through the elements of the VARRAY in original order
  FOR i IN 1 .. nums.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE('Element ' || i || ': ' || nums(i));
    -- Add each element to the total sum
    total := total + nums(i);
  END LOOP;

  -- Display the sum of all elements
  DBMS_OUTPUT.PUT_LINE('Sum of elements: ' || total);

  -- Reset total for reversed order sum
  total := 0;

  -- Display all elements in the collection in reverse order
  DBMS_OUTPUT.PUT_LINE('Elements in the collection (reversed order):');

  -- Loop through the elements of the VARRAY in reverse order
  FOR i IN REVERSE 1 .. nums.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE('Element ' || i || ': ' || nums(i));
    -- Add each element to the total sum
    total := total + nums(i);
  END LOOP;

  -- Display the sum of elements in reversed order
  DBMS_OUTPUT.PUT_LINE('Sum of elements in reversed order: ' || total);
END;
/
```

Output



Collections: Nested Table

```
DECLARE
  -- Declare a TABLE type named NumTable that can store multiple NUMBER values
  TYPE NumTable IS TABLE OF NUMBER;

  -- Declare a variable 'nums' of type NumTable and initialize it with three values
  nums NumTable := NumTable(5, 10, 15);

  -- Variables to store sum and product of elements
  total_sum NUMBER := 0;
  total_product NUMBER := 1;
BEGIN
  -- Delete the second element (creates a gap)
  nums.DELETE(2);

  -- Display all elements in the collection
  DBMS_OUTPUT.PUT_LINE('Elements in the collection:');

  -- Loop through the collection to process available elements
  FOR i IN 1 .. nums.LAST LOOP
    -- Check if the element exists (since DELETE creates gaps)
    IF nums.EXISTS(i) THEN
      DBMS_OUTPUT.PUT_LINE('Element ' || i || ': ' || nums(i));
      -- Add to sum
      total_sum := total_sum + nums(i);
      -- Multiply for product
      total_product := total_product * nums(i);
    END IF;
  END LOOP;

  -- Display the sum and product of available elements
  DBMS_OUTPUT.PUT_LINE('Sum of elements: ' || total_sum);
  DBMS_OUTPUT.PUT_LINE('Product of elements: ' || total_product);
END;
/
```

SQL*Plus

Nested Table

- Like an array but **dynamically resizable**.
- **Starts as dense**, can become **sparse** (gaps allowed).

Output

```
Elements in the collection:
Element 1: 5
Element 3: 15
Sum of elements: 20
Product of elements: 75
```

Records in PL/SQL

A **record** is a **group of related fields** that can store **different data types**.

Types of Records

1. **Table-based Record** (%ROWTYPE).
2. **User-defined Record** (TYPE).
3. **Cursor-based Record** (%ROWTYPE for cursor).

Table-Based Record
Uses %ROWTYPE to inherit structure from a database table.

```
DECLARE
  -- Declare a record variable to hold employee data
  emp_rec employees%ROWTYPE;
BEGIN
  -- Attempt to fetch the employee details based on employee_id
  SELECT * INTO emp_rec FROM employees WHERE employee_id = 100;

  -- Display employee details
  DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_rec.first_name || ' ' || emp_rec.last_name);
EXCEPTION
  -- Handle case where no matching employee is found
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error: No employee found with ID 100.');

  -- Handle case where multiple rows are returned (shouldn't happen for primary key search)
  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('Error: Multiple employees found with ID 100. Please check the data.');

  -- Catch all other unexpected errors
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Output

```
Employee Name: Alice
```

Records in PL/SQL : User-Defined Record

```
DECLARE
  -- Define a RECORD type to store employee details
  TYPE EmployeeRec IS RECORD (
    emp_id NUMBER,
    emp_name VARCHAR2(50),
    salary NUMBER
  );

  -- Declare a variable 'emp' of type EmployeeRec
  emp EmployeeRec;
BEGIN
  -- Assign values to the record fields
  emp.emp_id := 101;
  emp.emp_name := 'John Doe';
  emp.salary := 5000;

  -- Display the employee details
  DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp.emp_id);
  DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp.emp_name);
  DBMS_OUTPUT.PUT_LINE('Salary: ' || emp.salary);
EXCEPTION
  -- Handle unexpected errors
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

SQL*Plus

2.2 User-Defined Record
You can define custom record types.

Output

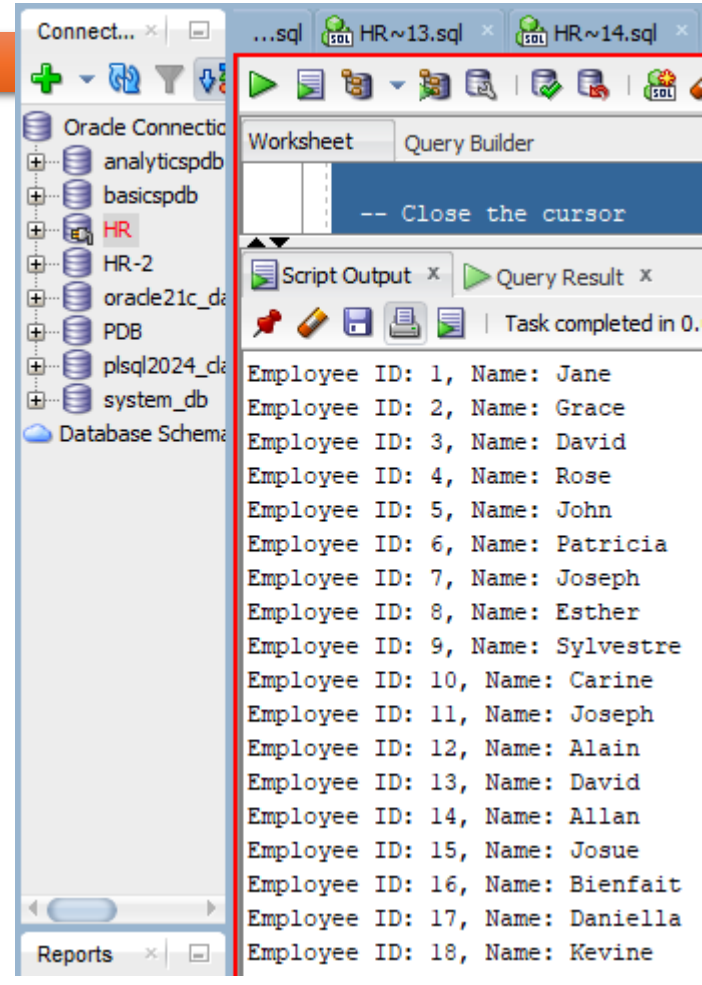
```
Employee ID: 101
Employee Name: John Doe
Salary: 5000
```

Records in PL/SQL : Cursor-Based Record

```
DECLARE
  -- Declare a cursor to fetch employee_id and first_name from employees table
  CURSOR c_emp IS SELECT employee_id, first_name FROM employees;

  -- Declare a record to store the fetched row
  emp_rec c_emp%ROWTYPE;
BEGIN
  -- Open the cursor
  OPEN c_emp;

  -- Fetch rows in a loop until all records are retrieved
  LOOP
    FETCH c_emp INTO emp_rec;

    -- Exit when no more data is found
    EXIT WHEN c_emp%NOTFOUND;

    -- Display the employee's first name
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_rec.employee_id || ', Name: ' || emp_rec.first_name);
  END LOOP;

  -- Close the cursor
  CLOSE c_emp;
EXCEPTION
  -- Handle cases where the cursor operation fails
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    IF c_emp%ISOPEN THEN
      CLOSE c_emp;
    END IF;
END;
/
```

2.3 Cursor-Based Record
Uses a cursor to fetch rows into a record.

Output

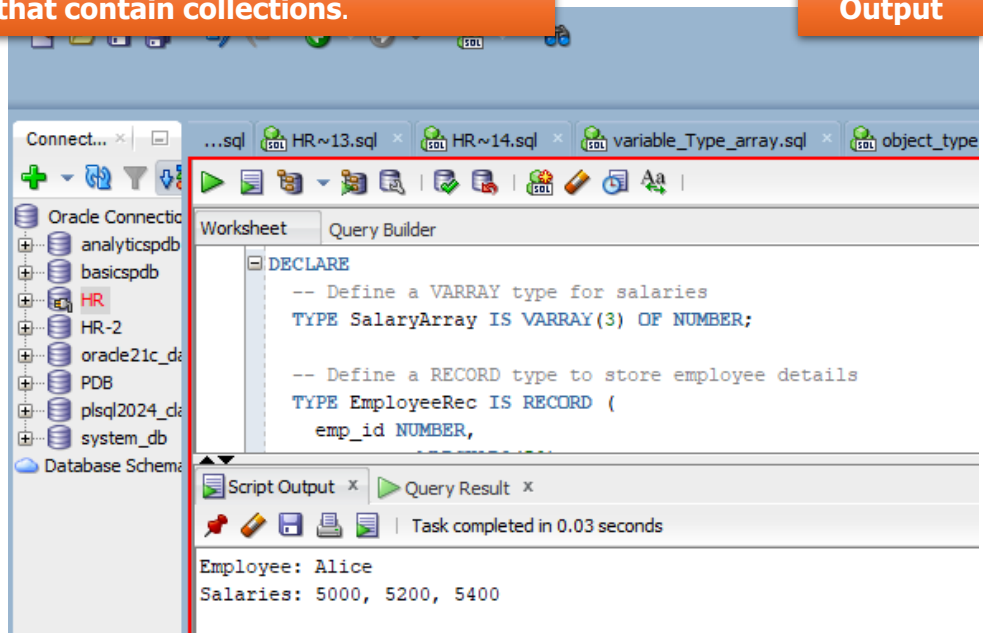


Using Collections and Records Together

You can create **collections of records** and **records that contain collections**.

```
DECLARE
  -- Define a VARRAY type for salaries
  TYPE SalaryArray IS VARRAY(3) OF NUMBER;

  -- Define a RECORD type to store employee details
  TYPE EmployeeRec IS RECORD (
    emp_id NUMBER,
    emp_name VARCHAR2(50),
    salaries SalaryArray  -- Use the predefined VARRAY type
  );

  -- Declare a variable 'emp' of type EmployeeRec
  emp EmployeeRec;
BEGIN
  -- Assign values to the record fields
  emp.emp_id := 101;
  emp.emp_name := 'Alice';
  emp.salaries := SalaryArray(5000, 5200, 5400);  -- Correct way to initialize VARRAY

  -- Display employee details
  DBMS_OUTPUT.PUT_LINE('Employee: ' || emp.emp_name);
  DBMS_OUTPUT.PUT_LINE('Salaries: ' || emp.salaries(1) || ', ' || emp.salaries(2) || ', ' || emp.salaries(3));

EXCEPTION
  -- Handle unexpected errors
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Output



Key Fixes & Enhancements:

1. Corrected VARRAY Initialization

- The SalaryArray type is now declared separately before using it inside EmployeeRec.
- emp.salaries := SalaryArray(5000, 5200, 5400); is the correct way to assign values.

2. Displays all salaries

- Instead of only showing salaries(1), it now prints all three salary values.

3. Exception Handling

- If an error occurs, it prints SQLERRM to help debug issues.

Question: PL/SQL example on VARRAYs

Question

You are tasked with writing a PL/SQL block to manage the salaries of a department with a fixed number of employees. The department has a maximum of 5 employees, and you need to store their monthly salaries in a VARRAY.
Write a PL/SQL block that:

1. Defines a VARRAY to store the salaries (with a maximum size of 5).
2. Assigns salary values to the VARRAY (for example, salaries of 5000, 6000, 7000, 8000, and 9000).
3. Displays the salary of each employee using a loop.

Hint: Use the DBMS_OUTPUT.PUT_LINE function to print the salaries.

```
DECLARE
    TYPE salary_array IS VARRAY(5) OF NUMBER;  -- A VARRAY that can hold up to 5 numbers
    v_salaries salary_array;  -- Declare the VARRAY type variable
BEGIN
    -- Assign values to the VARRAY elements
    v_salaries := salary_array(5000, 6000, 7000, 8000, 9000);

    -- Display the salaries
    FOR i IN 1..v_salaries.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE('Salary ' || i || ': ' || v_salaries(i));
    END LOOP;
END;
```

Output

```
Salary 1: 5000
Salary 2: 6000
Salary 3: 7000
Salary 4: 8000
Salary 5: 9000
```

Key Differences Between Collections and Records

Feature	Collections	Records
Elements Type	Same type	Different types
Indexing	Uses index (number or string)	Uses field names
Structure	Homogeneous	Heterogeneous
Example Usage	Storing multiple numbers or names	Storing employee details (ID, name, salary)

Conclusion

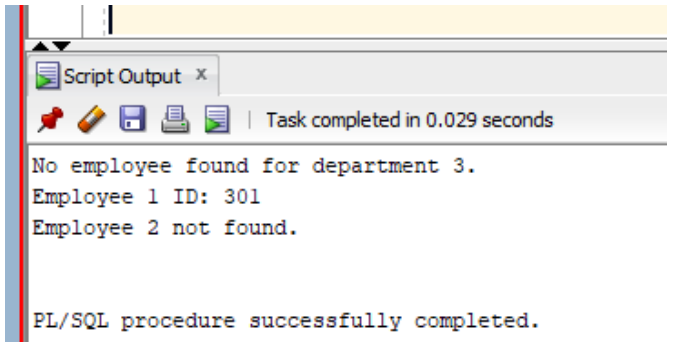
- **Collections** store multiple elements of the same type.
- **Records** store fields of different types.
- **Collections** can be **associative arrays, VARRAYs, or nested tables**.
- **Records** can be **table-based, user-defined, or cursor-based**.

Object Types

You can also define user-defined object types in PL/SQL, which are useful for more complex scenarios.
Example of using an object type:

```
-- Start of the PL/SQL Anonymous Block
DECLARE
    -- Declare a table type to hold employee IDs, indexed by an integer
    TYPE employee_ids_table IS TABLE OF EMPLOYEES.EMPLOYEE_ID%TYPE
INDEX BY BINARY_INTEGER;

    -- Declare a variable to store the employee IDs
    v_employee_ids employee_ids_table;
BEGIN
    -- Begin block to retrieve the first employee ID (department 1, first employee)
    BEGIN
        -- Select employee ID for department 1 and store it in v_employee_ids(1)
        SELECT employee_id
        INTO v_employee_ids(1)
        FROM employees
        WHERE department_id = 1 AND ROWNUM = 1;
    EXCEPTION
        -- Handle the case where no employee is found (i.e., no rows returned)
        WHEN NO_DATA_FOUND THEN
            -- Output message indicating no employee found for department 1
            DBMS_OUTPUT.PUT_LINE('No employee found for department 1.');
            -- Set v_employee_ids(1) to NULL (or a default value) in case of no data
            v_employee_ids(1) := NULL;
    END;

    -- Begin block to retrieve the second employee ID (department 3, second
employee)
    BEGIN
        -- Select employee ID for department 3 and store it in v_employee_ids(2)
        SELECT employee_id
        INTO v_employee_ids(2)
        FROM employees
        WHERE department_id = 3 AND ROWNUM = 2;
    EXCEPTION
        -- Handle the case where no employee is found (i.e., no rows returned)
        WHEN NO_DATA_FOUND THEN
            -- Output message indicating no employee found for department 3
             DBMS_OUTPUT.PUT_LINE('No employee found for department 3.');
            -- Set v_employee_ids(2) to NULL (or a default value) in case of no data
            v_employee_ids(2) := NULL;
    END;
```

```
    -- Check if employee 1 ID was found, and display the
result
    IF v_employee_ids(1) IS NOT NULL THEN
        DBMS_OUTPUT.PUT_LINE('Employee 1 ID: ' ||
v_employee_ids(1));
    ELSE
        -- If employee 1 was not found, output a different
message
        DBMS_OUTPUT.PUT_LINE('Employee 1 not found.');
    END IF;

    -- Check if employee 2 ID was found, and display the
result
    IF v_employee_ids(2) IS NOT NULL THEN
        DBMS_OUTPUT.PUT_LINE('Employee 2 ID: ' ||
v_employee_ids(2));
    ELSE
        -- If employee 2 was not found, output a different
message
        DBMS_OUTPUT.PUT_LINE('Employee 2 not found.');
    END IF;

-- End of the PL/SQL Anonymous Block
END;
/
```



You can also define user-defined object types in PL/SQL, which are useful for more complex scenarios.
Example of using an object type:

```
-- Start of the PL/SQL Anonymous Block
DECLARE
   -- Declare a table type to hold employee IDs, indexed by an integer
   TYPE employee_ids_table IS TABLE OF EMPLOYEES.EMPLOYEE_ID%TYPE
INDEX BY BINARY_INTEGER;

   -- Declare a variable to store the employee IDs
   v_employee_ids employee_ids_table;
BEGIN
   -- Begin block to retrieve the first employee ID (department 1, first employee)
   BEGIN
      -- Select employee ID for department 1 and store it in v_employee_ids(1)
      SELECT employee_id
      INTO v_employee_ids(1)
      FROM employees
      WHERE department_id = 1 AND ROWNUM = 1;
   EXCEPTION
      -- Handle the case where no employee is found (i.e., no rows returned)
      WHEN NO_DATA_FOUND THEN
         -- Output message indicating no employee found for department 1
         DBMS_OUTPUT.PUT_LINE('No employee found for department 1.');
         -- Set v_employee_ids(1) to NULL (or a default value) in case of no data
         v_employee_ids(1) := NULL;
   END;

   -- Begin block to retrieve the second employee ID (department 3, second
employee)
   BEGIN
      -- Select employee ID for department 3 and store it in v_employee_ids(2)
      SELECT employee_id
      INTO v_employee_ids(2)
      FROM employees
      WHERE department_id = 3 AND ROWNUM = 2;
   EXCEPTION
      -- Handle the case where no employee is found (i.e., no rows returned)
      WHEN NO_DATA_FOUND THEN
         -- Output message indicating no employee found for department 3
          DBMS_OUTPUT.PUT_LINE('No employee found for department 3.');
         -- Set v_employee_ids(2) to NULL (or a default value) in case of no data
         v_employee_ids(2) := NULL;
   END;
```

```
 -- Check if employee 1 ID was found, and display the
result
   IF v_employee_ids(1) IS NOT NULL THEN
      DBMS_OUTPUT.PUT_LINE('Employee 1 ID: ' ||
v_employee_ids(1));
   ELSE
      -- If employee 1 was not found, output a different
message
      DBMS_OUTPUT.PUT_LINE('Employee 1 not
found.');
   END IF;

   -- Check if employee 2 ID was found, and display the
result
   IF v_employee_ids(2) IS NOT NULL THEN
      DBMS_OUTPUT.PUT_LINE('Employee 2 ID: ' ||
v_employee_ids(2));
   ELSE
      -- If employee 2 was not found, output a different
message
      DBMS_OUTPUT.PUT_LINE('Employee 2 not
found.');
   END IF;

-- End of the PL/SQL Anonymous Block
END;
/
```

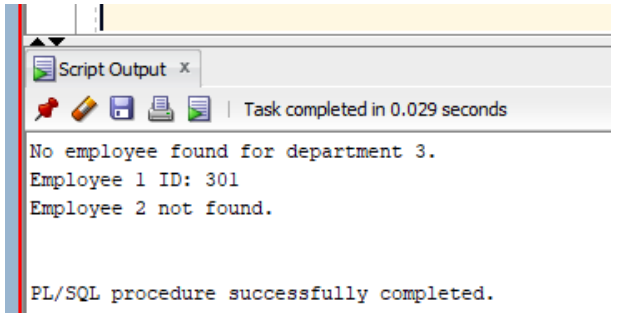
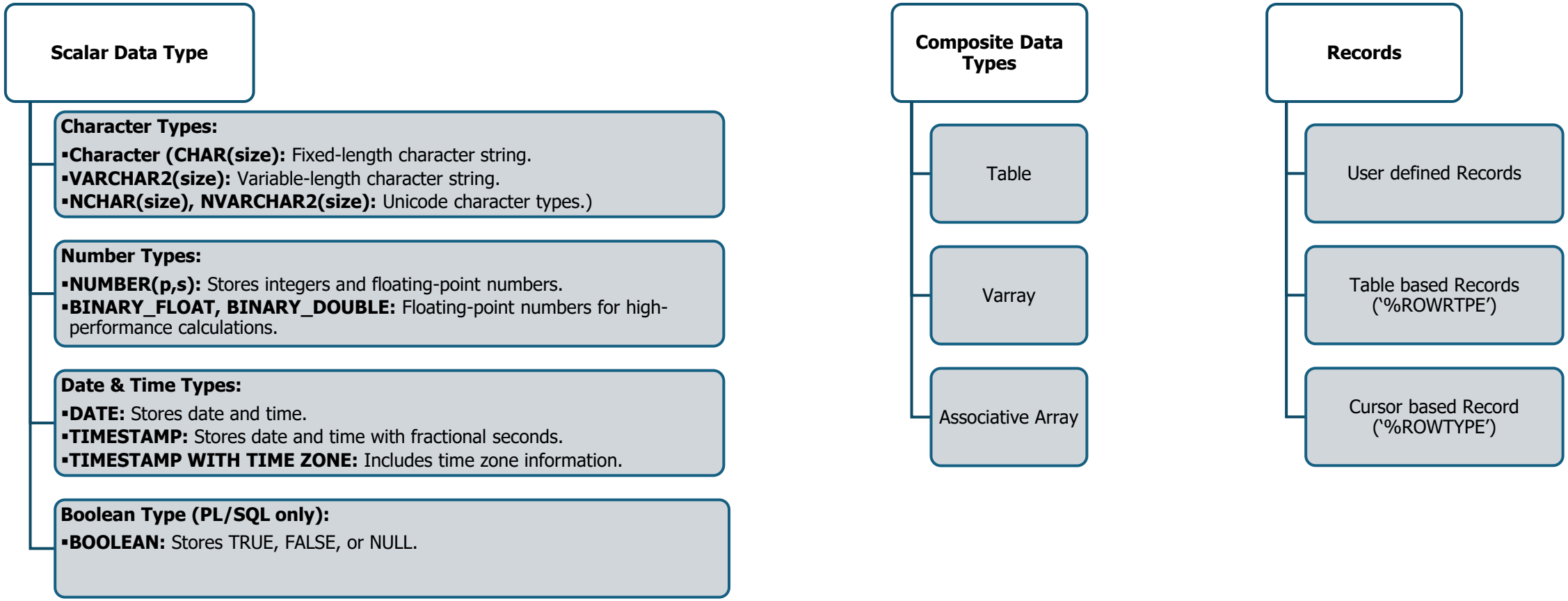


Diagram representing Scalar, Composite, and Record data types in Oracle



Keep Learning!